Optimization

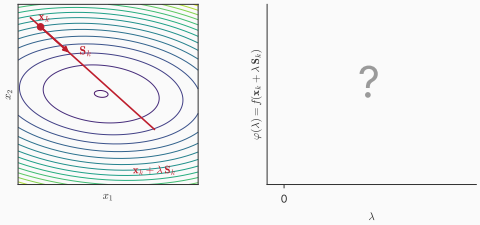
2.1 Univariate optimization: definition, classification and general remarks

José Firmino Aguilar Madeira

Chapter 2. Univariate optimization — why a single variable?

The problems we care about have *n* variables. Why study optimization in *one*?

Almost all methods of Ch. 3 do, at each iteration, $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{S}_k$: they choose a direction $\mathbf{S}_k$ and then how far to go, λ_k.



Chapter 2. Univariate optimization — why a single variable?

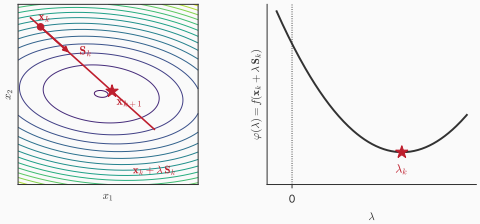
The problems we care about have *n* variables. Why study optimization in *one*?

Almost all methods of Ch. 3 do, at each iteration, $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{S}_k$: they choose a direction $\mathbf{S}_k$ and then how far to go, λ_k.

With $\mathbf{x}_k$ and $\mathbf{S}_k$ fixed, $\mathbf{x}_k + \lambda \mathbf{S}_k$ is a line, and along it

$$\varphi(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{S}_k)$$

is a function of one variable. Choosing λ_k means minimizing φ: the line search.



Chapter 2. Univariate optimization — why a single variable?

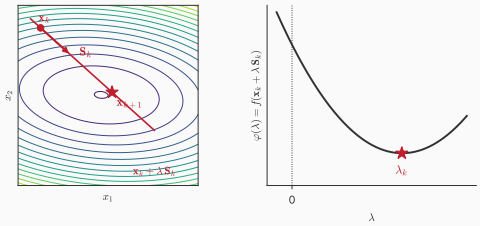
The problems we care about have *n* variables. Why study optimization in *one*?

Almost all methods of Ch. 3 do, at each iteration, $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{S}_k$: they choose a direction $\mathbf{S}_k$ and then how far to go, λ_k.

With $\mathbf{x}_k$ and $\mathbf{S}_k$ fixed, $\mathbf{x}_k + \lambda \mathbf{S}_k$ is a line, and along it

$$\varphi(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{S}_k)$$

is a function of one variable. Choosing λ_k means minimizing φ: the line search.



In 1D we decide how far to go. In *n*D we must first decide where to go (Ch. 3).
And the line search is repeated at every iteration: in Ch. 3 it is often most of the cost.

The problem: minimum value and minimizer

A univariate optimization problem: $\min_{a \le x \le b} f(x)$ (the only constraint is the interval).

There are two possible answers:

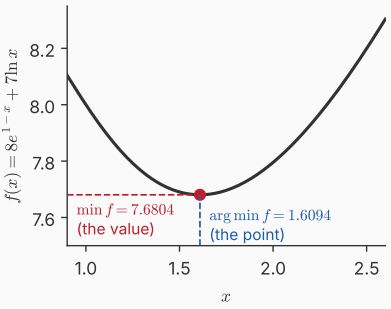
$$\text{minimum value:} \quad f^* = \min_{a \le x \le b} f(x)$$

$$\text{minimizer:} \quad x^* = \arg\min_{a \le x \le b} f(x)$$

f^* is a value of f; x^* is the point where it is attained: $f(x^*) = f^*$. In the example: $\min f = 7.6804$, $\arg\min f = 1.6094$.

In the line search what matters is the point: $\lambda_k = \arg\min_{\lambda > 0} \varphi(\lambda)$.

If there are several minimizers, arg min denotes the set of them; in this chapter there will be only one.



$f(x) = 8e^{1-x} + 7\ln x$ on $[1, 2]$: the example that runs through the chapter (2019 first test).

What can we know about f?

What information do we have? How much does it cost to obtain? What can we guarantee with it?
(the question that runs through the course)

$f(x)$ only	order 0	$\longrightarrow$	2.2 golden section: *eliminate* interval; 2.3 Powell: *interpolate*
$f(x)$ and $f'(x)$	order 1	$\longrightarrow$	2.4 bisection: *locate* the zero of f'
f, f' and f''	order 2	$\longrightarrow$	2.5 Newton: *linearize* f'
only f, but we want f', f''		$\longrightarrow$	2.6 finite differences: *approximate* f', f''

More information: potentially faster (linear, superlinear, quadratic)

but it costs: derivatives are costly, or unreliable if f is noisy; and it may make the method less robust (no guaranteed bracketing)

The cost of a method is measured in evaluations of f (and of f', f''), not in arithmetic operations. We count separately n_f (evaluations of f), n_g (of f'; in $\mathbb{R}^n$, gradients) and n_H (of f''; Hessians).

Iterative methods: when to stop?

Except in trivial cases, the minimum is not obtained in a single step. An iterative method:

1. starts from an initial approximation x_0 (or from an interval $[a_0, b_0]$);
2. generates a sequence $x_1, x_2, \ldots$ in which each term uses information on f at the previous ones — only at x_k (Newton), or at several: successive quadratic interpolation (Powell's method) uses three;
3. stops when a criterion is satisfied.

Usual stopping criteria

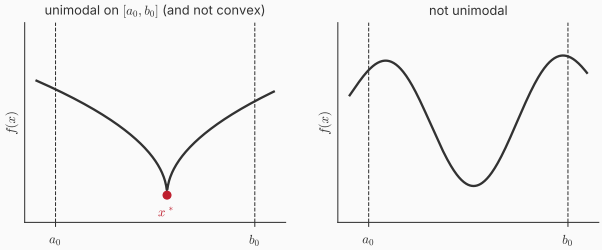
$$\frac{|x_{k+1} - x_k|}{\max(1, |x_k|)} \leq \varepsilon_x, \quad |f(x_{k+1}) - f(x_k)| \leq \varepsilon_f, \quad |f'(x_k)| \leq \varepsilon_g, \quad b_k - a_k \leq \varepsilon_x, \quad k \geq k_{\max}.$$

Always use $k_{\max}$ (it protects against methods that do not converge). Dividing by $\max(1, |x_k|)$ makes the criterion relative when $|x_k|$ is large and absolute near zero; golden section uses the length $b_k - a_k$; bisection (2.4), $|f'(z)| \leq \varepsilon$.

Unimodal functions

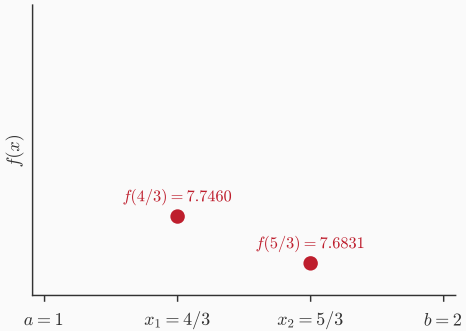
Definition

f is unimodal on $[a_0, b_0]$ if it has one and only one local minimum x^* in that interval: it is strictly decreasing on $[a_0, x^*]$ and strictly increasing on $[x^*, b_0]$.



It requires neither continuity, nor derivatives, nor convexity — the "down, then up" shape is enough. Every strictly convex function is unimodal; the converse is false (on the left: unimodal, not convex and not differentiable at x^*).

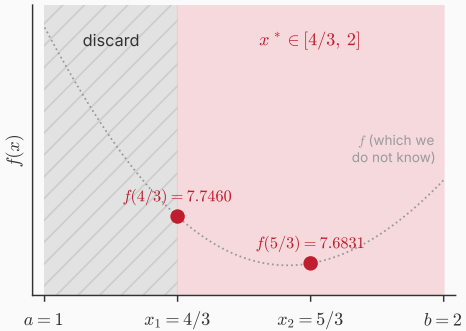
Only two values of f: what can we throw away?



f is unimodal on $[1, 2]$, but we do not know it: we only have $f(4/3) = 7.7460$ and $f(5/3) = 7.6831$.

Which part of the interval are we sure we do not need?

Only two values of f: what can we throw away?

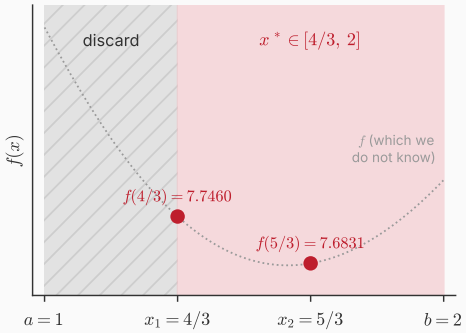


f is unimodal on $[1, 2]$, but we do not know it: we only have $f(4/3) = 7.7460$ and $f(5/3) = 7.6831$.

Which part of the interval are we sure we do not need?

Since $f(4/3) > f(5/3)$, the minimum is not in $[1,\ 4/3)$: if it were, f would increase from $4/3$ to $5/3$. Hence $x^* \in [4/3,\ 2]$ — one third of the interval discarded with two evaluations and *without derivatives*.

Only two values of f: what can we throw away?



f is unimodal on $[1, 2]$, but we do not know it: we only have $f(4/3) = 7.7460$ and $f(5/3) = 7.6831$.

Which part of the interval are we sure we do not need?

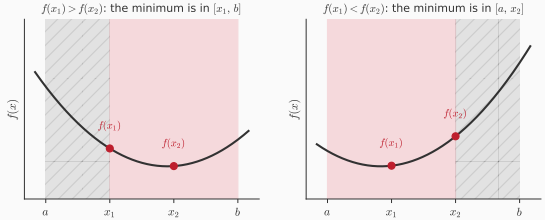
Since $f(4/3) > f(5/3)$, the minimum is not in $[1,\ 4/3)$: if it were, f would increase from $4/3$ to $5/3$. Hence $x^* \in [4/3,\ 2]$ — one third of the interval discarded with two evaluations and *without derivatives*.

And $x_2 = 5/3$ stays inside the new interval, with f already computed.

If we repeat this many times, where should x_1, x_2 be placed to reuse an evaluation? $\rightarrow$ 2.2

The general rule: interval elimination

With f unimodal on $[a, b]$ and two interior points $x_1 < x_2$:

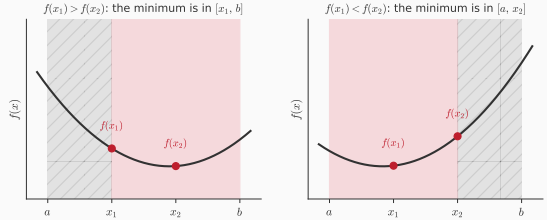


$$f(x_1) > f(x_2) \;\Rightarrow\; x^* \in [x_1, b], \qquad f(x_1) < f(x_2) \;\Rightarrow\; x^* \in [a, x_2], \qquad f(x_1) = f(x_2) \;\Rightarrow\; x^* \in [x_1, x_2].$$

Robust (guaranteed bracketing): the interval always contains x^*, and shrinks predictably. Only the order of the two values is used ($<$ or $>$).

The general rule: interval elimination

With f unimodal on $[a, b]$ and two interior points $x_1 < x_2$:



$$f(x_1) > f(x_2) \;\Rightarrow\; x^* \in [x_1, b], \qquad f(x_1) < f(x_2) \;\Rightarrow\; x^* \in [a, x_2], \qquad f(x_1) = f(x_2) \;\Rightarrow\; x^* \in [x_1, x_2].$$

Robust (guaranteed bracketing): the interval always contains x^*, and shrinks predictably. Only the order of the two values is used ($<$ or $>$).

What if we also used their *magnitude*? With three values of f a parabola can be fitted to estimate where the minimum is $\rightarrow$ 2.3 (Powell).

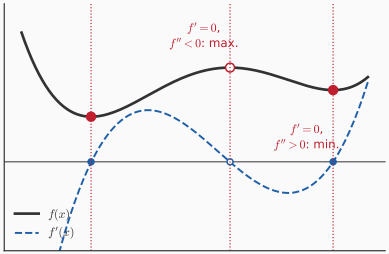
With derivatives: looking for the zero of f'

If f is differentiable, at an interior minimum $f'(x^*) = 0$ (necessary condition); if moreover $f''(x^*) > 0$, x^* is a strict local minimum (sufficient condition).

Instead of minimizing f, we can look for the zero of f':

- **bisection** on f' (2.4): the *sign* of f' tells on which side the zero is. It keeps an interval with $f'(a) < 0 < f'(b)$ (f' continuous);
- **Newton** on f' (2.5): the slope of f', that is f'', lets us *estimate where* the zero is: $x_{k+1} = x_k - \dfrac{f'(x_k)}{f''(x_k)}$.

Caution: $f' = 0$ also at maxima and inflection points — check f'' ($f'' = 0$ is inconclusive).

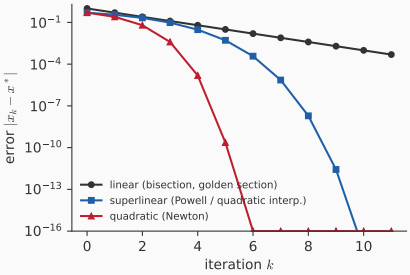


Rate of convergence

Let $e_k = |x_k - x^*|$. The convergence is called

- **linear** if $e_{k+1} \le c\, e_k$, $0 < c < 1$ — it gains a fixed number of digits per iteration. In interval methods it is the *length* L_k that decreases linearly ($L_{k+1} = 0.5 L_k$ in bisection, $0.618 L_k$ in golden section) and $e_k \le L_k/2$;
- **superlinear** if $e_{k+1}/e_k \to 0$ (quadratic interpolation: order ≈ 1.3);
- **quadratic** if $e_{k+1} \le c\, e_k^2$ — near x^*, the number of correct digits *tends to double* per iteration (Newton).

The price of speed: Newton needs f'' and x_0 close by; under their respective assumptions, bisection and golden section have guaranteed convergence.



Chapter map: five ideas

Method	Information	Idea	Guarantee / speed
2.2 Golden section	f	eliminate interval	robust (f unimodal); linear, 0.618 per evaluation
2.3 Powell	f	interpolate a parabola	fast near the minimum (superlinear); on its own, no guarantee
2.4 Bisection	f'	locate the zero of f'	robust (single bracketed zero); linear, 0.5
2.5 Newton	f', f''	linearize f'	local quadratic; may diverge
2.6 Finite differences	f	approximate f', f''	truncation error + round-off error

In practice they are combined: a robust method (guaranteed bracketing) locates the minimum, a fast one refines it.

golden section + parabolic interpolation → `fminbnd`

→ line search of Ch. 3 (steepest descent, damped Newton, CG)

Ch. 2 is not just "earlier material": it explains a component that works *inside* the algorithms of Ch. 3.

KEY TAKEAWAYS — 2.1

1. In $\mathbb{R}^n$, in direction-based methods, one chooses a direction and then how far to go: $\lambda_k = \arg\min_{\lambda>0} \varphi(\lambda)$, with $\varphi(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{S}_k)$ — a problem in one variable.
2. $\min f$ is the minimum *value*; $\arg\min f$ is the *point* where it is attained.
3. Methods differ in the information they use (f only, f', f''), in what it costs and in what it guarantees: more information may give faster convergence (linear $\rightarrow$ superlinear $\rightarrow$ quadratic), but it costs and may make the method less robust (no guaranteed bracketing).
4. With f unimodal, two values of f are enough to discard part of the interval (only their order is used); with derivatives, minimizing f becomes looking for the zero of f' ($f' = 0$ also at maxima: check f'').

Next (2.2): where to place x_1 and x_2 so that each reduction of the interval costs a single evaluation of f?

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2nd ed., PHI Learning, 2012 — Ch. 2.
- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Ch. 3, line search.

Resources

- MATLAB: `fminbnd`.